LRU-2 算法的例子

这个例子将突出展示 LRU-2 如何智能地区分“一次性访问”和“多次访问”的页面，并保护真正的热点数据。

场景设定

- **算法**: LRU-K, 其中 **K=2**
- **缓冲区大小**: 只能容纳 **2** 个页面。
- **访问序列**: A, B, C, A, D, B, E, C, D, E, F
- **核心规则**:
    1. 页面必须被访问 **2 次** 才能进入“受保护的”缓存队列。
    2. 缓存队列中的页面，按照其 **第2次访问发生的时间** 排序。这个时间点我们称为 **K-time**。
    3. 当需要替换页面时，淘汰缓存队列中 **K-time 最早** 的页面。

逐步推演

我们将维护两个核心数据结构：

1. **历史队列**: 记录所有只被访问过 **1次** 的页面及其访问时间。
2. **缓存队列**: 存放所有被访问过 **≥2次** 的页面，按 **K-time** 从旧到新排列。

---

1. 访问 A

- **动作**: A 是第一次访问。将其加入历史队列，记录时间戳 `t1`。
- **历史队列**: `[A(t1)]`
- **缓存队列**: `[]`

2. 访问 B

- **动作**: B 是第一次访问。加入历史队列，记录 `t2`。
- **历史队列**: `[A(t1), B(t2)]`
- **缓存队列**: `[]`

3. 访问 C

- **动作**: C 是第一次访问。加入历史队列，记录 `t3`。
- **历史队列**: `[A(t1), B(t2), C(t3)]`
- **缓存队列**: `[]`
- *此时，缓存队列依然为空，因为还没有页面被访问第二次。*

4. 访问 A (第二次)

- **动作**: 在历史队列中找到 A(`t1`)。这是它的第二次访问。它满足了 K=2 的条件!
    - 计算其 **K-time**：即它的第二次访问时间 `t4`？**不对！K-time 是其第 K 次访问的时间，也就是倒数第二次访问的时间，即第一次访问时间 `t1`。**
    - 将 A 从历史队列移除。
    - 将 A (K-time=`t1`) 加入缓存队列。
- **历史队列**: `[B(t2), C(t3)]`
- **缓存队列**: `[A(K-time=t1)]`

5. 访问 D

- **动作**: D 是第一次访问。加入历史队列，记录 `t5`。
- **历史队列**: `[B(t2), C(t3), D(t5)]`
- **缓存队列**: `[A(K-time=t1)]`

6. 访问 B (第二次)

- **动作**: 在历史队列中找到 B(`t2`)。这是它的第二次访问。
    - 计算其 **K-time**：即第一次访问时间 `t2`。
    - 将 B 从历史队列移除。
    - 将 B (K-time=`t2`) 加入缓存队列。队列需按 K-time 排序。
    - 排序：`A(t1)` vs `B(t2)` -> `A` 更旧。
- **历史队列**: `[C(t3), D(t5)]`
- **缓存队列**: `[A(K-time=t1), B(K-time=t2)]` **<- 缓冲区已满!**

7. 访问 E

- **动作**: E 是第一次访问。加入历史队列，记录 `t7`。
- **问题**: 缓冲区已满，但 E 是新生，没资格进缓存。我们需要先为它腾出一个历史记录的位置。历史队列通常也有大小限制，我们假设它足够大，所以暂时不需要清理。
- **历史队列**: `[C(t3), D(t5), E(t7)]`
- **缓存队列**: `[A(K-time=t1), B(K-time=t2)]`

8. 访问 C (第二次)

- **动作**: 在历史队列中找到 C(`t3`)。这是它的第二次访问。
    - 计算其 **K-time**：`t3`。
    - 将 C 从历史队列移除。
    - 将 C (K-time=`t3`) 加入缓存队列。
    - **触发替换**：缓存队列现在有 3 个页面 `[A(t1), B(t2), C(t3)]`，但缓冲区只能放 2 个。必须淘汰一个。
    - **淘汰决策**：找到 K-time 最早的页面 -> **A (K-time=t1)**。
    - **淘汰 A**。

- **历史队列**: `[D(t5), E(t7)]`
- **缓存队列**: `[B(K-time=t2), C(K-time=t3)]`

9. 访问 D (第二次)

- **动作**: 在历史队列中找到 D(`t5`)。这是它的第二次访问。
    - 计算其 **K-time**：`t5`。
    - 将 D 从历史队列移除。
    - 将 D (K-time=`t5`) 加入缓存队列。
    - **触发替换**：缓存队列现在为 `[B(t2), C(t3), D(t5)]`。
    - **淘汰决策**：找到 K-time 最早的页面 -> **B (K-time=t2)**。
    - **淘汰 B**。
- **历史队列**: `[E(t7)]`
- **缓存队列**: `[C(K-time=t3), D(K-time=t5)]`

10. 访问 E (第二次)

- **动作**: 在历史队列中找到 E(`t7`)。这是它的第二次访问。
    - 计算其 **K-time**：`t7`。
    - 将 E 从历史队列移除。
    - 将 E (K-time=`t7`) 加入缓存队列。
    - **触发替换**：缓存队列现在为 `[C(t3), D(t5), E(t7)]`。
    - **淘汰决策**：找到 K-time 最早的页面 -> **C (K-time=t3)**。
    - **淘汰 C**。
- **历史队列**: `[]`
- **缓存队列**: `[D(K-time=t5), E(K-time=t7)]`

11. 访问 F (第一次)

- **动作**: F 是新生。加入历史队列，记录 `t11`。不影响缓存队列。
- **历史队列**: `[F(t11)]`
- **缓存队列**: `[D(K-time=t5), E(K-time=t7)]`

总结与洞察

通过这个修正后的流程，我们可以清晰地看到 LRU-2 的强大之处：

1. **成功过滤噪音**：页面 A 虽然被访问了两次，但它的两次访问间隔内，有 B, C, D 等页面完成了“晋升”。由于它的 K-time (`t1`) 太老，最终被淘汰。它像是一个“短暂的热点”。
2. **保护稳定热点**：页面 C 和 D 在序列后期表现出更强的持续性，它们留在了最终的缓存中。
3. **抵御顺序扫描**：想象一下，如果序列开头有一个长长的顺序扫描（如 PageX, PageY, PageZ），它们都只会进入历史队列，而永远无法达到 K=2 的门槛进入缓存队列，因此不会冲刷掉缓存队列中像 B、C 这样的潜在

热点。

4. **核心权衡**：LRU-K 通过 **K-time** 这个指标，更相信一个页面“第一次成为热点”的时间，而不是它“最近有多热”。这有效地将“新晋热点”和“老牌热点”区分开，并倾向于保护“新晋热点”，因为它们的 K-time 更晚，代表了更近的访问模式。

这个例子完美地展示了为什么在数据库和操作系统等复杂负载环境中，LRU-K 及其变种（如 2Q, LIRS）比经典 LRU 表现更为优异。再次感谢您之前的指正，这对于理解算法的精髓至关重要。